

Chapter 1: Python, NumPy & Digital Image Fundamentals

Learning Objectives

- Understand how a computer represents grayscale and color images.
- Learn the Python syntax needed to begin computer-vision programming.
- Understand NumPy arrays, image shape, data type, indexing and slicing.
- Read, display, crop, resize, modify and save images.
- Understand RGB versus BGR, bit depth, dynamic range, sampling and quantization.

1. What is a Digital Image?

A digital image is a numerical representation of a scene. When we look at a photograph, we see objects, colors, edges and textures. A computer does not directly understand these concepts. It first receives the image as numbers arranged in a regular grid.

For a grayscale image, each grid location contains one number describing brightness. We can write a grayscale image mathematically as $I(x, y)$, where x and y identify a spatial location and $I(x, y)$ is the intensity measured at that location.

In an 8-bit grayscale image, the stored intensity is usually between 0 and 255. A value of 0 is displayed as black, 255 as white, and values between them appear as different shades of gray.

A color image normally stores several measurements at every spatial location. In an RGB image, these are the red, green and blue components. Therefore, a color pixel is not just one number; it is a small vector such as $[R, G, B]$.

This leads to the most important programming idea in this chapter: an image can be treated as an array. A grayscale image is usually a two-dimensional array of size Height x Width, while an RGB image is commonly a three-dimensional array of size Height x Width x 3.

A tiny numerical example

```
I = [  
    [0, 50, 100],  
    [150, 200, 250],  
    [30, 80, 120]  
]
```

This 3 x 3 matrix can be interpreted as a 3 x 3 grayscale image. The value 0 is very dark, 250 is very bright, and the other values create intermediate shades.

2. Python Environment for Computer Vision

Python is a general-purpose programming language. Computer vision becomes convenient in Python because mature libraries already provide efficient numerical operations, plotting tools and standard vision algorithms.

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

Line-by-line / step-by-step explanation

1. ``import`` tells Python to load a library so that its functions can be used in the current program.
2. ``numpy`` is imported as ``np``. NumPy provides fast numerical arrays and matrix operations. The short name ``np`` is a standard convention.
3. ``matplotlib.pyplot`` is imported as ``plt``. It is used to display images, graphs and experimental results.
4. ``cv2`` is the Python interface to OpenCV. OpenCV contains many image-processing and computer-vision algorithms.

If the packages are not installed, use:

```
pip install opencv-python numpy matplotlib
```

The installation command is normally run in a terminal or command prompt, not inside an ordinary Python script.

3. Python Fundamentals Needed for Computer Vision

3.1 Variables

```
width = 640
height = 480
channels = 3

print(width)
print(height)
print(channels)
```

Line-by-line / step-by-step explanation

1. ``width = 640`` creates a variable named `width` and stores the integer 640 in it.
2. ``height = 480`` stores the image height.
3. ``channels = 3`` represents a three-channel color image.
4. ``print(...)`` asks Python to show a value in the output area.

Expected output

```
640
480
3
```

3.2 Basic data types

```
x = 10
y = 3.14
name = "Computer Vision"
flag = True

print(type(x))
print(type(y))
```

```
print(type(name))
print(type(flag))
```

Line-by-line / step-by-step explanation

1. ``10`` is an integer, so `x` has type ``int``.
2. ``3.14`` contains a decimal part, so `y` has type ``float``.
3. Text placed inside quotation marks is a string (``str``).
4. ``True`` and ``False`` are Boolean values (``bool``).
5. ``type(variable)`` asks Python which data type is currently stored.

Expected output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

4. Python Lists Versus NumPy Arrays

```
pixels_list = [10, 20, 30]
print(pixels_list * 2)

pixels_array = np.array([10, 20, 30])
print(pixels_array * 2)
```

Line-by-line / step-by-step explanation

1. `[10, 20, 30]` creates an ordinary Python list.
2. Multiplying a Python list by 2 repeats the list; it does not multiply every number.
3. ``np.array(...)`` converts the values into a NumPy numerical array.
4. Multiplying a NumPy array by 2 performs element-by-element numerical multiplication. This behavior is much more useful for image processing.

Expected output

```
[10, 20, 30, 10, 20, 30]
[20 40 60]
```

5. NumPy Arrays as Images

```
image = np.array([
    [0, 50, 100],
    [150, 200, 250],
    [30, 80, 120]
])

print(image)
print(image.shape)
print(image.ndim)
```

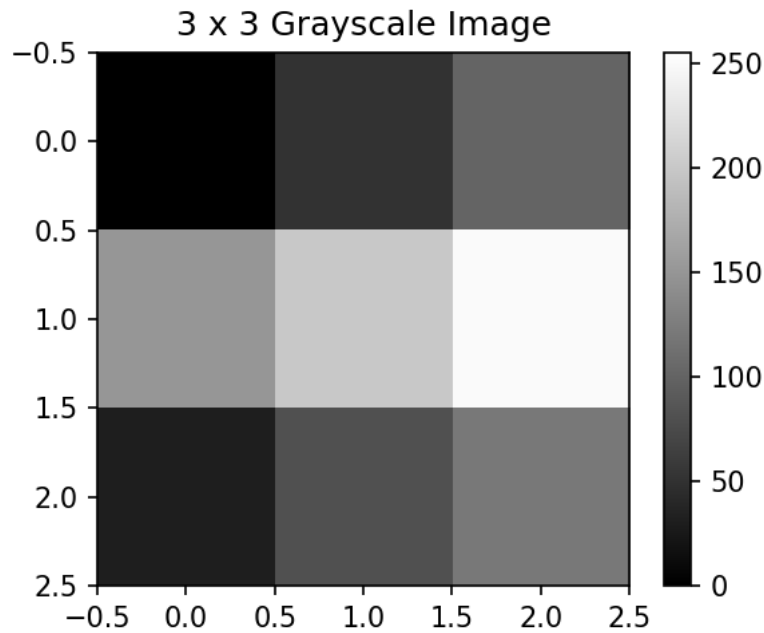
Line-by-line / step-by-step explanation

1. ``np.array`` creates a two-dimensional numerical array from the nested Python lists.

2. Each inner list becomes one row of the array.
3. `print(image)` displays the stored numerical values.
4. `image.shape` returns the size of each array dimension. Here the result is (3, 3).
5. `image.ndim` returns the number of dimensions. A grayscale matrix like this has two dimensions.

Expected output

```
[[ 0  50 100]
 [150 200 250]
 [ 30  80 120]]
(3, 3)
2
```



Expected visualization of the 3 x 3 grayscale matrix.

6. Image Coordinate System and Indexing

This point is extremely important. In mathematics we often describe an image location as (x, y), where x moves horizontally and y moves vertically. NumPy, however, accesses a two-dimensional array as [row, column]. Row corresponds approximately to y and column to x.

```
pixel = image[1, 2]
print(pixel)
```

Line-by-line / step-by-step explanation

1. `image[1, 2]` means row index 1 and column index 2.
2. Python indexing begins at zero. Therefore row 1 is the second row and column 2 is the third column.
3. The value stored there is 250.

Expected output

250

A useful memory rule is: NumPy image access is normally `image[y, x]`. Later, OpenCV may return geometric points as `(x, y)`, so always check which convention a function uses.

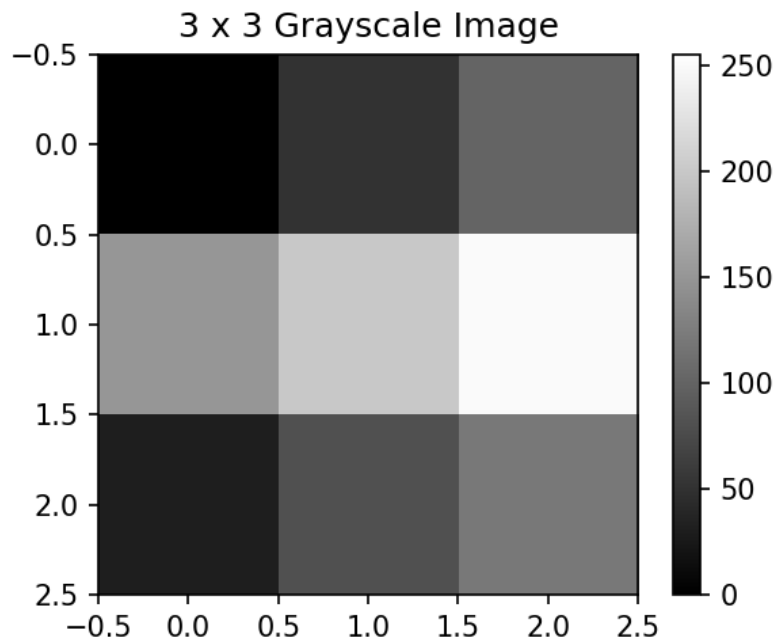
7. Visualizing an Image

```
plt.imshow(image, cmap="gray", vmin=0, vmax=255)
plt.colorbar()
plt.title("3 x 3 Grayscale Image")
plt.show()
```

Line-by-line / step-by-step explanation

1. `plt.imshow(image, ...)` tells Matplotlib to display the array as an image.
2. `cmap='gray'` selects a grayscale display map.
3. `vmin=0` and `vmax=255` fix the displayed intensity range. Without this, Matplotlib may automatically stretch the available values.
4. `plt.colorbar()` adds a scale showing how numerical intensity maps to displayed brightness.
5. `plt.title(...)` adds a title.
6. `plt.show()` renders the figure.

Expected output



Expected output: darker pixels correspond to smaller numerical values and brighter pixels to larger values.

8. Creating Synthetic Images

8.1 Black image

```
black_image = np.zeros((256, 256), dtype=np.uint8)

plt.imshow(black_image, cmap="gray", vmin=0, vmax=255)
plt.show()
```

Line-by-line / step-by-step explanation

1. `np.zeros((256,256))` creates 256 rows and 256 columns, all initialized to zero.`
2. `dtype=np.uint8` stores every value as an unsigned 8-bit integer.`
3. Since every pixel equals zero, the displayed image is completely black.

Expected output

Expected visual output: a 256 x 256 completely black square.

8.2 White and gray images

```
white_image = np.ones((256, 256), dtype=np.uint8) * 255
gray_image = np.ones((256, 256), dtype=np.uint8) * 128
```

Line-by-line / step-by-step explanation

1. `np.ones` creates an array filled with the value 1.`
2. Multiplying the array by 255 changes every value to 255, producing white.
3. Multiplying by 128 produces a middle-gray image.

Expected output

```
white_image: all pixel values = 255
gray_image: all pixel values = 128
```

9. Understanding uint8, Bit Depth and Dynamic Range

A data type tells Python how a number is stored. The type `uint8` means unsigned 8-bit integer. 'Unsigned' means negative values are not represented. Eight bits can encode $2^8 = 256$ different values, giving the range 0 to 255.`

Bit depth and spatial resolution are different. Bit depth tells us how finely intensity can be represented. Spatial resolution tells us how densely the scene is sampled across space. A 16-bit image can still have low spatial resolution, and an 8-bit image can have very high spatial resolution.

```
print(gray_image.dtype)
print(np.iinfo(np.uint8))
```

Line-by-line / step-by-step explanation

1. `gray_image.dtype` reports the array's stored numerical type.`
2. `np.iinfo(np.uint8)` displays the representable integer range for uint8.`

Scientific cameras may use 10-, 12-, 14- or 16-bit measurements. Therefore, never assume that every image uses the range 0-255.

10. Creating an Intensity Gradient

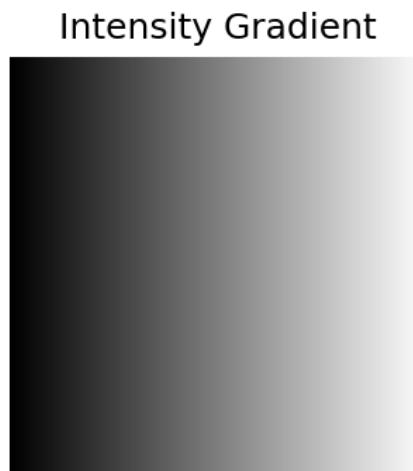
```
gradient = np.linspace(0, 255, 256)
gradient = np.tile.gradient, (256, 1))

plt.imshow(gradient, cmap="gray", vmin=0, vmax=255)
plt.title("Intensity Gradient")
plt.show()
```

Line-by-line / step-by-step explanation

1. `np.linspace(0,255,256)` creates 256 evenly spaced values beginning at 0 and ending at 255.
2. The first result is a one-dimensional row of values.
3. `np.tile(gradient, (256,1))` repeats that row 256 times vertically.
4. The resulting 2-D array has a gradual horizontal transition from black to white.

Expected output



Expected output: a smooth intensity ramp from black on the left to white on the right.

11. Reading a Real Image with OpenCV

```
img = cv2.imread("image.jpg")

if img is None:
    print("Image could not be loaded")
else:
    print("Image loaded successfully")
    print("Shape:", img.shape)
    print("Data type:", img.dtype)
    print("Minimum:", img.min())
    print("Maximum:", img.max())
```

Line-by-line / step-by-step explanation

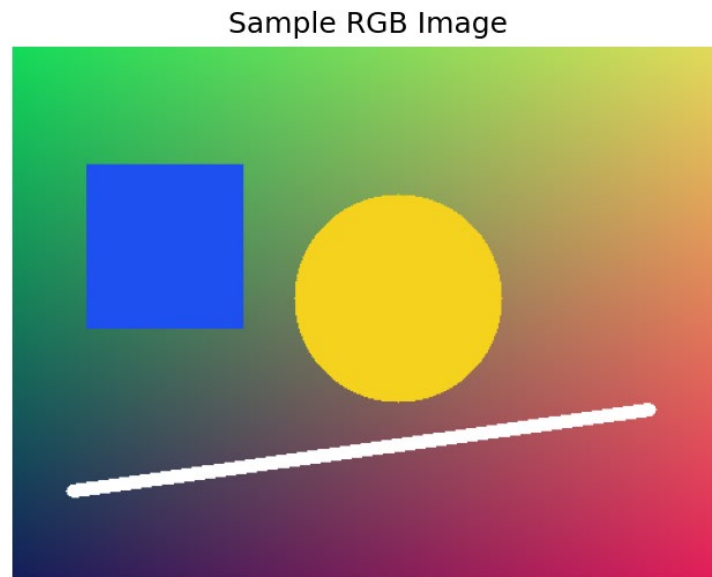
1. `cv2.imread('image.jpg')` asks OpenCV to read the file from the specified path.
2. If the path is wrong or the file cannot be decoded, OpenCV commonly returns `None`.

3. ``if img is None:`` checks for that failure condition before later code tries to use the image.
4. ``else:`` runs only when the image was loaded.
5. ``shape`` reports height, width and channels for a normal color image.
6. ``dtype`` reports the numerical storage type.
7. ``min()`` and ``max()`` show the observed intensity range in the loaded image.

For the reproducible sample image used in this chapter, an equivalent expected output is:

Expected output

```
Image loaded successfully
Shape: (360, 480, 3)
Data type: uint8
Minimum: 0
Maximum: 255
```



Example of the color image used for the remaining expected-output figures.

12. Important: OpenCV Uses BGR

OpenCV conventionally reads a color image in BGR order: Blue, Green, Red. Matplotlib expects RGB: Red, Green, Blue. If an OpenCV image is displayed directly with Matplotlib, red and blue may appear exchanged.

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

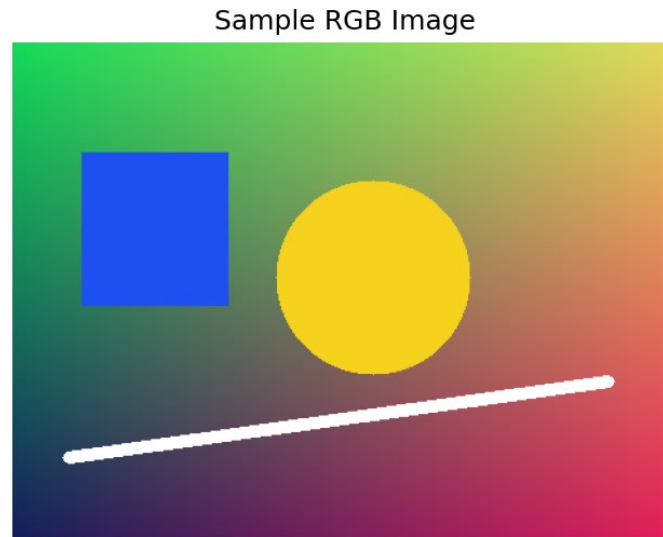
plt.imshow(img_rgb)
plt.title("RGB Image")
plt.axis("off")
plt.show()
```

Line-by-line / step-by-step explanation

1. ``cv2.cvtColor`` converts an image from one color representation to another.

2. ``cv2.COLOR_BGR2RGB`` specifically swaps the BGR interpretation into RGB order.
3. The converted array can then be displayed correctly with Matplotlib.
4. ``plt.axis('off')`` hides axis ticks because they are not needed for ordinary image viewing.

Expected output



Expected output: the image appears with its intended red, green and blue colors.

13. Extracting Image Dimensions

```
height, width, channels = img_rgb.shape

print("Height:", height)
print("Width:", width)
print("Channels:", channels)
```

Line-by-line / step-by-step explanation

1. ``img_rgb.shape`` returns three values for a color image.
2. Python allows the three returned values to be unpacked directly into height, width and channels.
3. For the sample image the values are 360, 480 and 3.

Expected output

```
Height: 360
Width: 480
Channels: 3
```

14. Accessing Individual Pixels

```
pixel = img_rgb[100, 200]
print(pixel)

R = pixel[0]
G = pixel[1]
B = pixel[2]
```

```
print("Red:", R)
print("Green:", G)
print("Blue:", B)
```

Line-by-line / step-by-step explanation

1. `img_rgb[100,200]` selects one spatial location: row 100, column 200.
2. Because this is a color image, the result contains three values rather than one.
3. After RGB conversion, element 0 is red, element 1 is green and element 2 is blue.
4. The three assignments make those values easier to inspect individually.

The exact numerical values depend on the image being used. This is normal: expected output for image-processing programs is often image-dependent.

15. Separating RGB Channels

```
R = img_rgb[:, :, 0]
G = img_rgb[:, :, 1]
B = img_rgb[:, :, 2]

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(R, cmap="gray")
plt.title("Red Channel")

plt.subplot(1, 3, 2)
plt.imshow(G, cmap="gray")
plt.title("Green Channel")

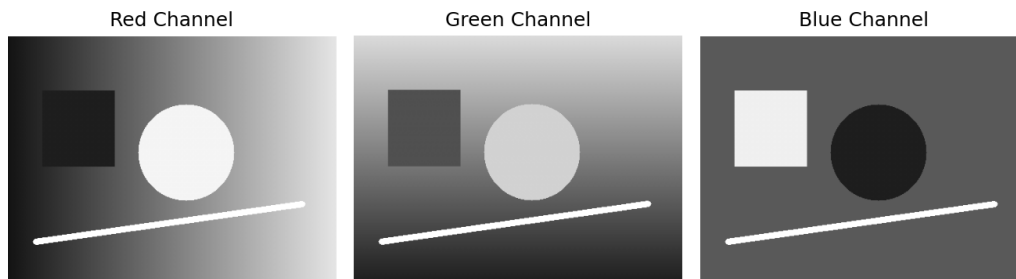
plt.subplot(1, 3, 3)
plt.imshow(B, cmap="gray")
plt.title("Blue Channel")

plt.show()
```

Line-by-line / step-by-step explanation

1. ``:`` means take all available indices along that dimension.
2. `img_rgb[:, :, 0]` therefore means all rows, all columns, channel 0.
3. The same logic extracts green and blue channels.
4. `plt.figure(figsize=(12,4))` creates a wider figure.
5. `subplot(1,3,1)` means one row, three columns of plots, and select the first plot.
6. Each channel is displayed using grayscale so brightness directly represents that channel's numerical strength.

Expected output



Expected output: three grayscale images showing the spatial contribution of the red, green and blue channels.

16. NumPy Slicing and Cropping

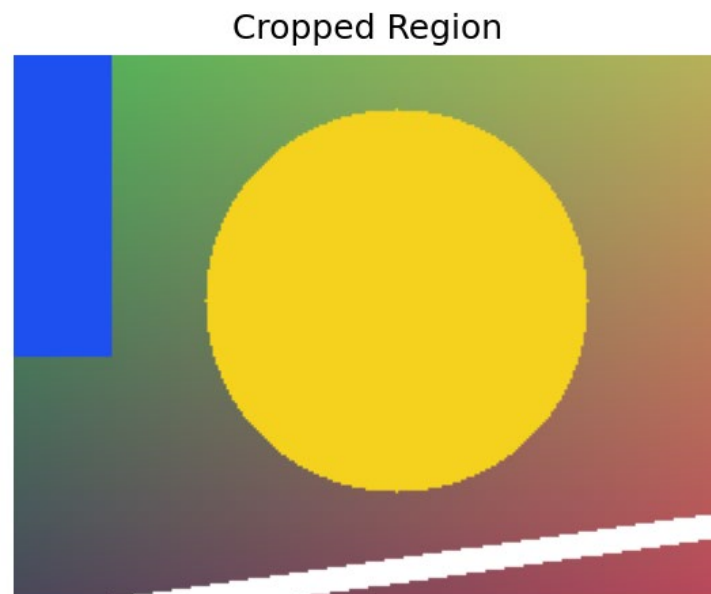
```
crop = img_rgb[80:280, 120:380]
```

```
plt.imshow(crop)
plt.title("Cropped Region")
plt.show()
```

Line-by-line / step-by-step explanation

1. ``80:280`` selects rows starting at 80 and stopping before 280.
2. ``120:380`` selects columns starting at 120 and stopping before 380.
3. The selected rectangular portion becomes a new array view/region.
4. Cropping is therefore fundamentally an indexing operation, not a special computer-vision algorithm.

Expected output



Expected output: only the selected rectangular region of the original image is displayed.

17. Modifying Pixels

```
modified = img_rgb.copy()
modified[100:200, 100:300] = 255

plt.imshow(modified)
plt.show()
```

Line-by-line / step-by-step explanation

1. ``copy()`` creates an independent copy so the original image is preserved.
2. The slice selects a rectangular region.
3. Assigning the scalar 255 to a three-channel slice broadcasts 255 to R, G and B.
4. R=G=B=255 is white, so the selected region becomes white.

Expected output

Expected visual output: the original image with a white rectangle covering rows 100-199 and columns 100-299.

18. Resizing an Image

```
resized = cv2.resize(img_rgb, (300, 200))

print(resized.shape)

plt.imshow(resized)
plt.title("Resized Image")
plt.show()
```

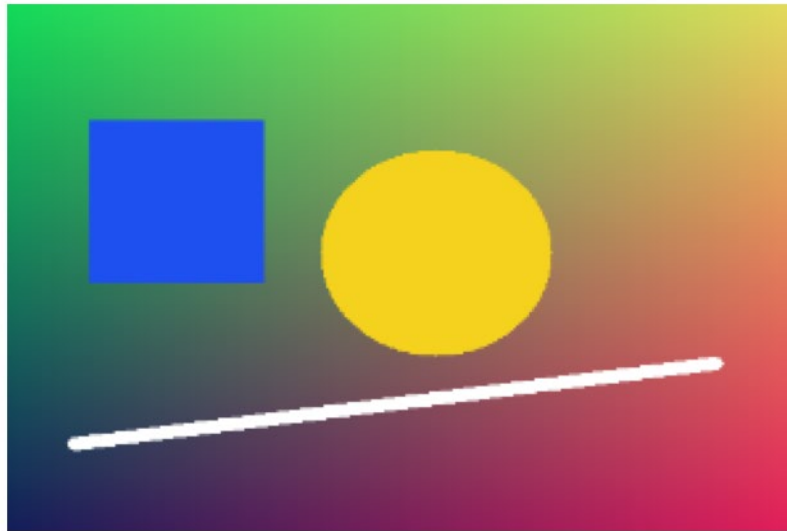
Line-by-line / step-by-step explanation

1. ``cv2.resize`` changes the spatial dimensions.
2. OpenCV specifies the requested size as ``(width, height)``, so ``(300,200)`` means width 300 and height 200.
3. The resulting NumPy shape is therefore ``(200,300,3)`` because NumPy reports height first.
4. Resizing requires interpolation because output sample positions generally do not coincide exactly with input pixel positions.

Expected output

```
(200, 300, 3)
```

Resized Image: 300 x 200



Expected visual output after resizing to width 300 and height 200.

19. Converting RGB to Grayscale

Grayscale conversion reduces three color measurements to one intensity/luminance-like measurement. It is not generally a simple average of R, G and B. Standard conversions use different weights because the channels do not contribute equally to perceived brightness.

```
gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

print(gray.shape)

plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.show()
```

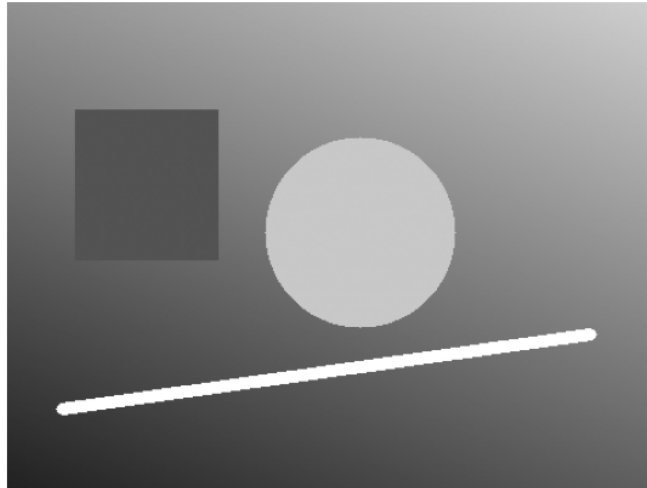
Line-by-line / step-by-step explanation

1. `COLOR_RGB2GRAY`` converts the three-channel RGB image into a single-channel grayscale image.
2. The output shape now contains only height and width because there is no separate color-channel dimension.
3. `cmap='gray'`` is used when displaying the resulting 2-D intensity array.

Expected output

```
(360, 480)
```

Grayscale Image



Expected output: a single-channel grayscale version of the sample color image.

20. Sampling

A real scene varies continuously across space, but a digital sensor measures the scene only at discrete locations. This process is called spatial sampling. If the sampling density is too low relative to scene detail, high-frequency information can be lost or aliased.

Downsampling deliberately reduces the number of spatial samples. Upsampling later creates a larger array, but ordinary interpolation cannot guarantee recovery of information that was removed.

```
h, w = img_rgb.shape[:2]

small = cv2.resize(
    img_rgb,
    (w // 4, h // 4),
    interpolation=cv2.INTER_AREA
)

restored = cv2.resize(
    small,
    (w, h),
    interpolation=cv2.INTER_LINEAR
)

print("Original:", img_rgb.shape)
print("Small:", small.shape)
print("Restored:", restored.shape)
```

Line-by-line / step-by-step explanation

1. ``shape[:2]`` takes only height and width.
2. ``w//4`` and ``h//4`` use integer division to reduce each spatial dimension by four.
3. ``INTER_AREA`` is commonly suitable for shrinking images.
4. The second resize returns the array to the original width and height.

5. `INTER_LINEAR` estimates new values using linear interpolation.

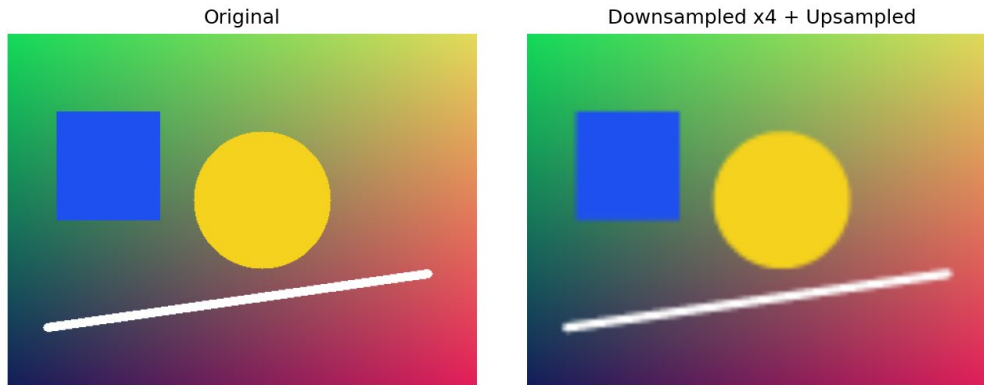
6. The restored image has the same array dimensions as the original, but fine information removed during downsampling is not automatically restored.

Expected output

Original: (360, 480, 3)

Small: (90, 120, 3)

Restored: (360, 480, 3)



Expected visual comparison: the restored image has the original dimensions but visibly reduced fine detail.

21. Quantization

Sampling discretizes spatial position. Quantization discretizes intensity. Imagine that a sensor produces a continuous measurement but the computer is allowed to store only a limited number of intensity values. The measurement must be assigned to one of those allowed levels.

Reducing the number of quantization levels makes the effect easy to see: smooth gradients become step-like and subtle intensity differences disappear.

```
gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

levels = 4
step = 256 / levels

quantized = np.floor(
    gray.astype(np.float32) / step
) * step

quantized = np.clip(quantized, 0, 255).astype(np.uint8)

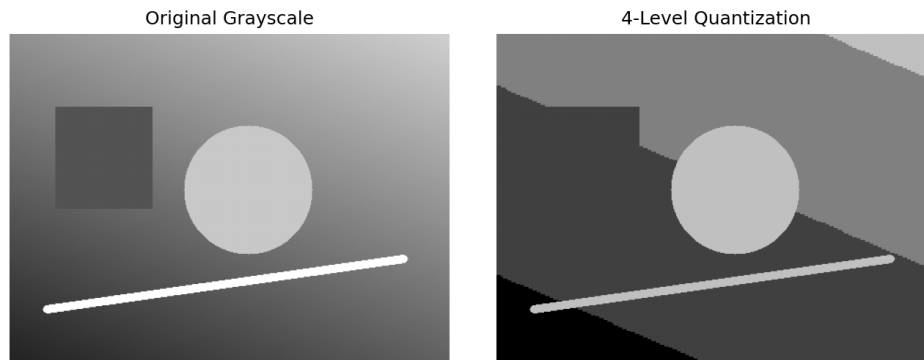
plt.imshow(quantized, cmap="gray", vmin=0, vmax=255)
plt.title("4-Level Quantized Image")
plt.show()
```

Line-by-line / step-by-step explanation

1. `levels = 4` requests only four output intensity groups.
2. `step = 256/4` gives a spacing of 64 intensity values.
3. `astype(np.float32)` converts uint8 values to floating point before division.

4. Dividing by `step` expresses each pixel in units of quantization intervals.
5. `np.floor` rounds downward to an integer interval index.
6. Multiplying by `step` maps the interval index back to an intensity value.
7. `np.clip` guarantees the final values remain within 0-255.
8. `astype(np.uint8)` converts the result back to an 8-bit image for ordinary display/storage.

Expected output



Expected output: only a few gray levels remain, producing visible intensity banding/posterization.

22. Normalization and Floating-Point Representation

```
img_float = img_rgb.astype(np.float32) / 255.0
```

```
print(img_float.dtype)
print(img_float.min())
print(img_float.max())
```

Line-by-line / step-by-step explanation

1. `astype(np.float32)` changes the storage type from integer to floating point.
2. Dividing by 255 maps the conventional 8-bit range approximately from [0,255] to [0,1].
3. Floating-point images are convenient for filtering, optimization and deep-learning operations because fractional values can be represented.

Expected output

```
float32
0.0
1.0
```

A crucial habit is to know both the data type and numerical range. A float image in [0,1] and a uint8 image in [0,255] may look similar when displayed, but arithmetic on them behaves differently.

23. Basic Image Statistics

```
print("Minimum:", gray.min())
print("Maximum:", gray.max())
print("Mean:", gray.mean())
print("Standard deviation:", gray.std())
```

Line-by-line / step-by-step explanation

1. ``min()`` finds the darkest stored intensity in the image.
2. ``max()`` finds the brightest.
3. ``mean()`` computes the average intensity.
4. ``std()`` measures how widely intensities are spread around their mean.
5. These simple statistics are useful for checking data and will connect directly to histograms and contrast in Chapter 2.

The exact numbers depend on the chosen image, so the correct expected output is the presence of four numerical values rather than one universal set of numbers.

24. Saving Results

```
output_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
success = cv2.imwrite("output_image.png", output_bgr)

print(success)
```

Line-by-line / step-by-step explanation

1. Because OpenCV writing functions conventionally operate with BGR color order, the RGB working image is converted back to BGR.
2. ``cv2.imwrite`` writes the array to a file.
3. The function returns a Boolean indicating whether the save operation succeeded.

Expected output

```
True
```

For quantitative image-processing work, lossless formats such as PNG or TIFF are often preferable when repeated lossy JPEG compression could affect measurements.

25. Complete Chapter 1 Practice Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img_bgr = cv2.imread("image.jpg")
if img_bgr is None:
    raise FileNotFoundError("Check the image path.")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Inspect
print("Shape:", img_rgb.shape)
print("dtype:", img_rgb.dtype)
print("min/max:", img_rgb.min(), img_rgb.max())

# Grayscale
```

```

gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

# Central crop
h, w = img_rgb.shape[:2]
crop = img_rgb[h//4:3*h//4, w//4:3*w//4]

# Downsample and restore
small = cv2.resize(
    img_rgb, (w//4, h//4),
    interpolation=cv2.INTER_AREA
)
restored = cv2.resize(
    small, (w, h),
    interpolation=cv2.INTER_LINEAR
)

# Quantize to four levels
levels = 4
step = 256 / levels
quantized = np.floor(
    gray.astype(np.float32) / step
) * step
quantized = np.clip(
    quantized, 0, 255
).astype(np.uint8)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(restored)
plt.title("Downsampled + Upsampled")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(
    quantized, cmap="gray",
    vmin=0, vmax=255
)
plt.title("4-Level Quantization")
plt.axis("off")

plt.tight_layout()
plt.show()

```

This program deliberately combines the chapter's most important operations. A beginner should be able to explain every block before moving to Chapter 2.

Line-by-line / step-by-step explanation

1. The first three lines import the required libraries.

2. ``imread`` loads the image and the ``None`` check catches a bad path.
3. The BGR-to-RGB conversion prepares the image for Matplotlib.
4. The inspection block checks shape, data type and numerical range.
5. The grayscale conversion creates a 2-D intensity image.
6. The central crop calculates its coordinates from image dimensions instead of hard-coding them.
7. The downsampling block removes spatial samples; the second resize restores dimensions for comparison.
8. The quantization block reduces the available grayscale levels.
9. The final figure arranges four results in a 2 x 2 layout so the effects can be compared directly.

Expected visual output: a four-panel figure containing the original RGB image, grayscale image, downsampled-and-upsampled image, and four-level quantized image.

26. Exercises

- Create a 256 x 256 uint8 grayscale image in which the left half is black and the right half is white.
- Create a 256 x 256 image containing a centered 100 x 100 white square on a black background using NumPy slicing only.
- Load a color image and print its height, width, channels, data type, minimum, maximum, mean and standard deviation.
- Extract the red, green and blue channels and determine which has the highest mean intensity.
- Crop the central 50% of an image without hard-coding coordinates.
- Downsample an image by factors 2, 4 and 8 and restore each to the original dimensions. Compare the visible information loss.
- Quantize a grayscale image to 2, 4, 8, 16 and 256 levels.
- Normalize a uint8 image to [0,1], multiply by 0.5, convert back to uint8 and explain the brightness change.
- Modify a rectangular region of an RGB image so that it becomes pure red.
- Write a function named `describe_image(image)` that prints shape, dtype, minimum, maximum, mean and standard deviation.

27. Conceptual Questions

- What is the difference between sampling and quantization?
- Why does an RGB image normally have three dimensions in NumPy?
- Why can a resized low-resolution image have the same dimensions as the original but contain less information?
- What is the difference between NumPy indexing order and conventional (x,y) notation?
- Why should uint8 arithmetic be handled carefully?
- Why can an OpenCV image appear with incorrect colors in Matplotlib?
- What is the difference between bit depth and spatial resolution?
- Why is floating-point representation useful in image processing and deep learning?